EPFL - Computer Science

COM-402 Information Security and Privacy

Lecture 08 — 10/11/2025

Author: Jonathan Dutoit

Professor: Mathias Payer

Mobile Security

Learning goals

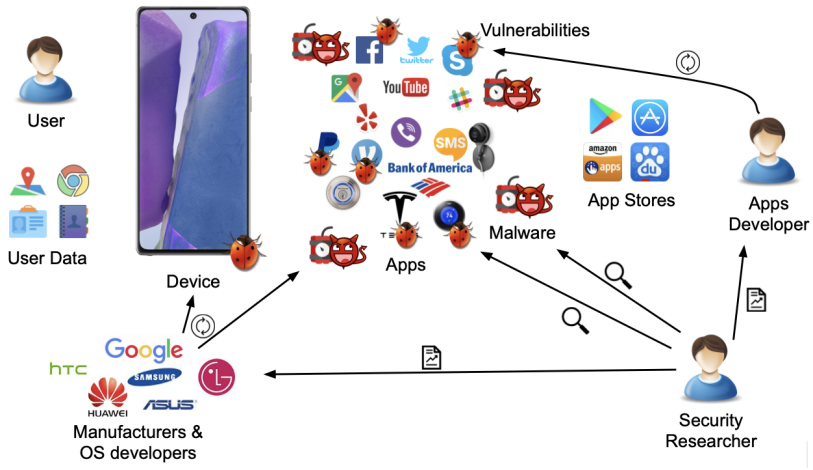
- Differentiate the security goals of mobile systems from desktop/server systems
- Awareness of security trade-offs for Android (vs. iOS)
- Understand security mitigations deployed by vendors

Mobile Security Motivation

Computing Revolutions

1. Data processing (in the 70ies)
2. Personal Computing (in the 80ies)
3. Internet (in the 90ies): Computers now exposed to everyone else
4. Mobile Computing (in the 00's): Mobile ecosystem keeps personal data
5. Social (in the 10's): Nightmare for privacy/personal data
6. AI? (20's): The web is dead: everything is in silos

Why is mobile security hard? On a phone, a bunch of software is running from multiple sources...



Types of Malware: Repackaging / Look-alike Malware

Download app from app market (e.g., Google Play Store)

Unpack and reverse engineer app ⇒ This is relatively easy on Android

Modify and add code

- Add ads
- Require more permissions than original app to steal user data
- Escalate privileges using exploits to deploy ransomware
- ...

Pack and publish modified version

Piracy of Digital Content

Steal digital content from content providers. Platform needs support for secure audio and video playback. Premium content must not leave the decide!

Streaming services usually require platform with digital rights management (DRM) solutions to stream premium content (e.g., 4K streams)

You can't record your screen while watching Netflix nor directly read the video files. That's because Netflix software is managed by a different part of the phone called **Trusted Execution Environment**

DRM Solution:

- Google Widevine
- Microsoft PlayReady

Broader: Political Instrument

Takeaways: mobile security is very important and try to avoid making enemies with money.

Android Ecosystem and Security

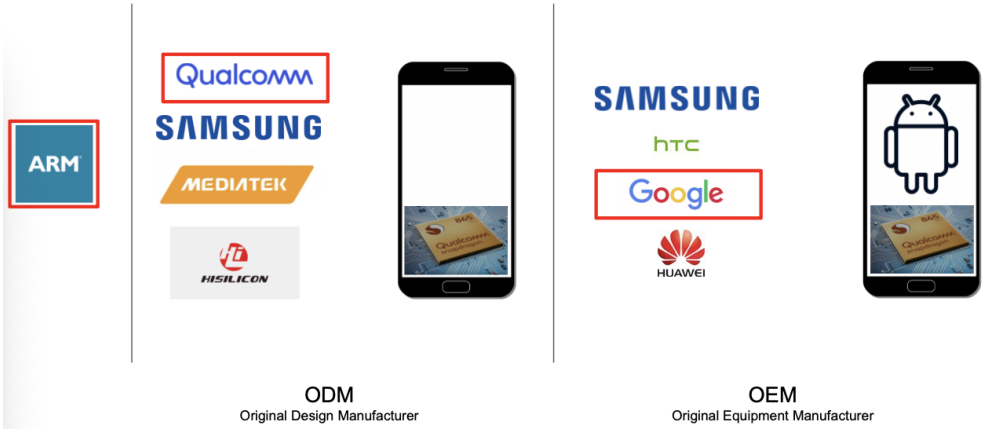
Android Open Source Project (AOSP)

Android is "**owned**" by Google. Code is made available through AOSP.

Original Equipment Manufacturer (OEM) customizes code to their hardware. OEM adds proprietary components (e.g., for hardware support). For instance if you buy Xiaomi or Pixel phone, they look very different!

Google Mobile Services (Google Play, YouTube, Google Maps, Gmail, ...) require OEM to comply with compatibility definition document

From Chip to Android Phone



The "Open" in AOSP

Q: "How does setting a PIN/pattern/password work?"

1. Determine component that implements feature
2. Figure out AOSP release
3. Look for component in AOSP
4. Read code until you are satisfied

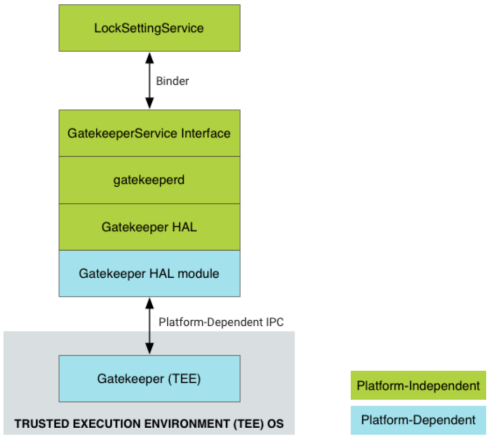
The "not-so-Open" in AOSP

Enrolling and verifying passwords is a security-sensitive feature

The backed of this feature is implemented by the Gatekeeper subsystem

This subsystem often uses the TEE and is usually platform-dependent (∼ proprietary)

- Guarantees stronger confidentiality
- Throttles consecutive failed attemps



Android Devices

Many different players: Many different hardware manufacturers / OEMs who customize their software/hardware stack

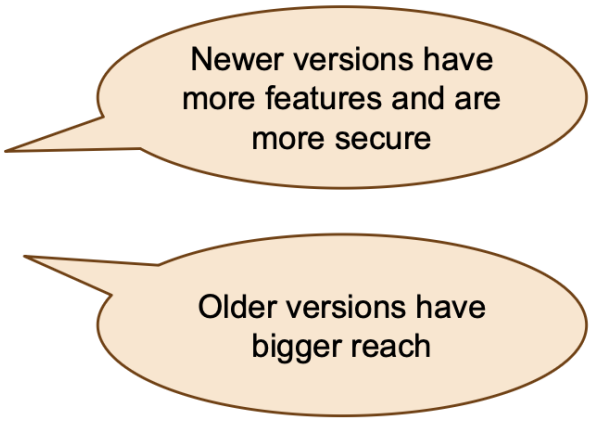
Many different Android versions

- Not all devices support all versions
- Some devices never get updates (tablets)
- Most devices get updates only for $\sim$ 3 years
- Android versions distribution secluded

What Android Version Would You Target?

If you're building your own application, you must consider trade-offs between security and reachability





Android 10 devices still being sold (6 years old)

Android Fragmentation

Fragmentation is the ***biggest challenger*** for Android security

Thousands of devices with custom hardware and software

Customization makes development and patching tricky

The Long Life of a Security Bug: Reporting

1. Discovery: Security researchers (Google's or external) finds a bug
2. Disclosure: Report the bug to Google
3. Reception:
   - [Hope the bug reaches the right Google employee]
   - Google acknowledges the bug and will work on a fix

The Long Life of a Security Bug: Fixing

(Responsible) Coordinated Disclosure

- [Wait n days, industry standard: 90 days]
- Publicly disclose bug

Security patch distribution

- Google distributes the patches to other manufactures
- Patch is published as part of the Monthly Security Bulletin

Supported Google devices get the patch. Vendors (e.g., Xiaomi) are advertised of found bugs. **Problem**: Vendors are always using a fork (own modifications)

The perils of customization

Google sends patch to third-party company X: Company X now needs to apply the patch to their custom AOSP fork

Is the patch compatible with X's customizations?

- What if the patch conflicts with code that was slightly modified?
- Would including the patch make everything unstable?
- Would it introduce backward compatibility problems? Run all the tests! but are the tests comprehensive enough?

Patch deployment can be risky

- 2B+ monthly active Android users. Broken patch can do damage.

Project Treble

Previously: no clear interface between Android OS framework and vendor implementation

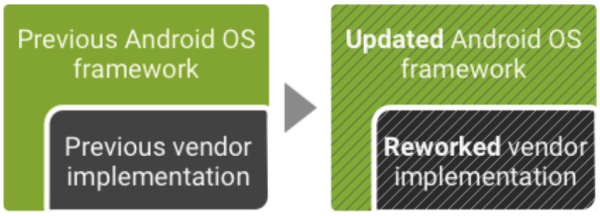
When the Android OS is update:

- Vendors lose time integrating the new patches
- This introduces significant costs, overhead and delays

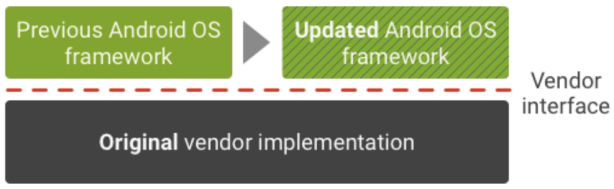
Project Treble helps defining a clear interface

Note: This is useful for non-security patches as well.

Up to Android 7.x:



In Android 8.0+ (with Project Treble):



Most Devices (Still) Don't Get Security Patches

Even with all this progress, many devices don't receive security updates

Most devices get support for only ∼3 years after launch (!= after you buy)

Some vendors (such as Google themselves) ***slowly*** extend support: Pixel 8 and 8 Pro will get updates for at least 7 years.

The Role of Security Researchers

Identify malware

- Once a new malware is found, it's added to blacklists
- Any app "similar" to the malware will be flagged and removed

Security vulnerabilities

- Once they are found, they are reported to Google
- Monetary rewards for security bugs: "Bug Bounties"

Remember: attackers will also target anything not included in the bug bounties!

Attacker Goals vs. Security Researchers

Security researchers want to

- Identify malware (add to disallow-lists, flag apps "similar enough" to listed malware apps) Report security vulnerabilities for bug bounties

Attackers want to

- In general: run code on your device and do XYZ

Two main important topics:

- Malware analysis, detection, containment
- Vulnerability analysis, detection, prevention, patching

Android Apps

Android apps in a Nutshell

Apps are written in Java, C/C++, Kotlin, Rust, ...

Android apps "live" in the Android context

- Android framework exposes a gigantic number of APIs
- APIs somewhat equivalent to "library functions"

APIs are useful for

- Apps to interact with the "external world" (via
- The Android framework to interact with the Android app, e..g., where does the execution start?
- To extend Java SDK APIs: Many APIs from the Java world are part of the Android APIs

The size of documentation for Android Framework APIs is **huge**!

Example of APIs: access to GPS/location info

```
LocationListener list = new MyLocationListener();
list.requestLocationUpdates(LocationManager.GPS_PROVIDER, 5000,
  10, locationListener);

class MyLocationListener implements LocationListener {
  @Override
  public void onLocationChanged(Location loc) {
    Toast.makeText("Location" + loc.getLatitude() +
    ", " + loc.getLongitude());
  }
}
```

Package Name

Each app has a package name, e.g., "com.facebook.katana"

The package name acts as a "unique" identifier across the system

Constraints: package name needs to be *unique*

- across apps installed on a specific Android device
- across the Google Play Store

Basics on Android apps

There's no "main" function (like in C programs)

The user interacts via the Graphical UI

- Many types of UI Widgets: EditText, Buttons, ...
- No command line interface

Many APIs are ”event-driven”

- You register a ”listener” X
- X’s callback is invoked later on

Apps are built as a combination of ”components”

The Four Component Types

- Activity: entry point for interacting with the user. It represents a single screen with a user interface. You can have many: each of them defines a UI. Also, you can define which one is the ”main” one. If app allows it, an external app can start these activates at will.
- Service: meant to perform an action in the background for some period of time, regardless of what the user is doing in foreground (the user could be witching between activities) e.g., Spotify
- Broadcast Receiver: meant to respond system-wide events. They have a well-defined entry point as well. The system can deliver these events even to apps that are currently not running. Examples: battery charging, SMS is received
- Content Provider: manage a shared set of app data. High-level API to access data so that other apps and services can query / interact with. Most often based on SQLite database (file-based). Example: calendar events, contacts list, settings

Each component type has its own life cycle

Communication Between Components: Intents

How can these components talk to each other?

Android-defined objects that encode an ”Intent” (a fancy IPC message)

Examples:

- ”Sharing content to an app”, done via Intents
- ”Choosing a file”, send an Intent to FilePicker Activity
- ”Open a webpage”, send an Intent to the default Browser Acitivty
- ”Take a picture”, send an Intent to the Camera Activity

The Manifest File

The most important file of an Android app

It specifies all key information needed by the framework to run the app:

- Package name
- Components app consists of
- Intent filters
- Main Activity $->$ main() entrypoint

- Permissions

Android Studio

Central/recommended IDE for Android app development

Common IDE features like

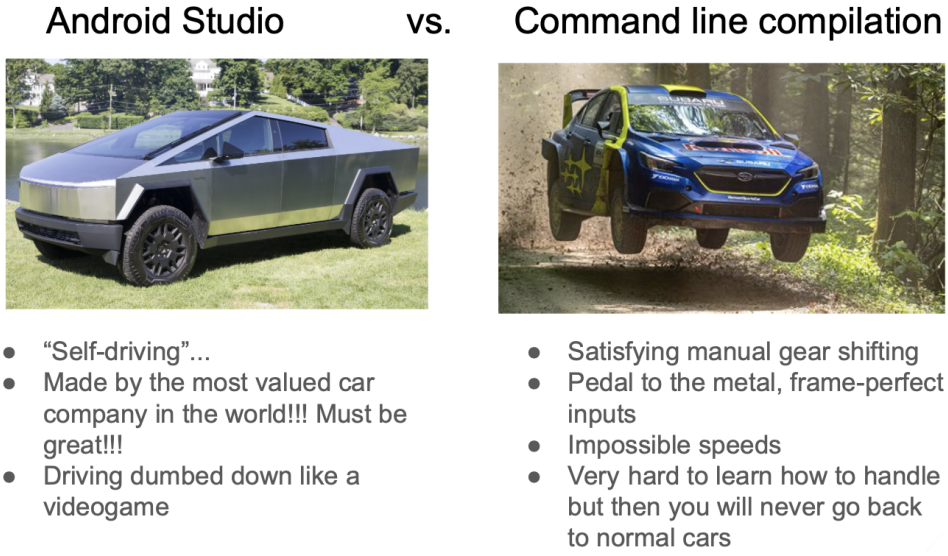
- Syntax highlighting
- Go to Definition/Usage
- Tooling for compilation, packaging, deployment
- Tons of plugins: vim bindings!

Android-specific features like

- Android Virtual Device (AVD) manager
- And much, much more...

Some people hate it: too slow, crashes, quirks, etc...

You can **compile it from the command line**! So you don't need Android Studio.



App Security

From the Eyes of an app

Android is based on Linux. Each app has its own Linux user ID

> There are ways to setup apps so that they share the user ID.

Each app lives in its own security *sandbox*

- Standard Linux process isolation
- Restricted file system permissions

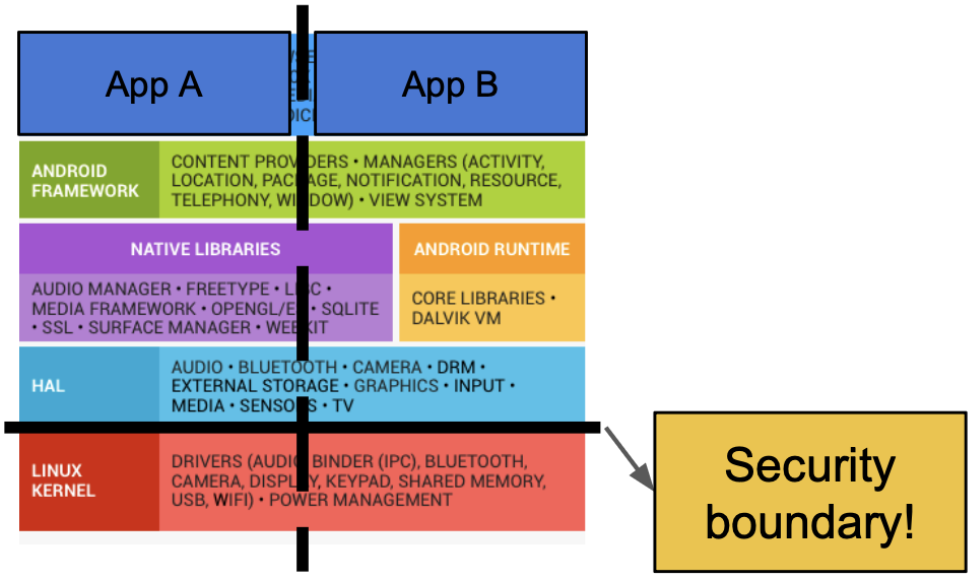
For **App installation**, the Android framework creates a new Linux user with a new user ID*

Each app is given a private direction

- Also called "Internal Storage"
- No other app can access it

> There are ways to setup app so that they share the user ID

Android Framework Architecture



Be aware: Malware is often written in native code without using any of Android Framework available tools!

syscalls (Digression)

Traditional OSes (like Windows, Linux, Android) have two worlds: User-space vs. Kernel-space. Kernel-space commands are called **syscall**

- User-space: where user processes and apps live: They can't do much by themselves (the JVM is also part of the user-space)
- Kernel-space is where the actual OS lives: The OS is the God of your machine and information

Some apps are different from the others: they are called **Managers**: *They run as "privileged" users/apps!*. Examples: LocationManager, TelephonyManager, etc...

Example: getLastLocation()

App invokes Android API:

- `LocationManager.getLastLocation()`
- We are still within the app's sandbox!

Actual implementation of the privileged API:

- `LocationManagerService.getLastLocation()`
- We are in a "privileged" service

How do we go from one side to the other one?

Crossing the Bridge

Binder: one of the main Android's "extensions" over Linux

It allows for

- Remote Procedure call (RCP)
- Inter-Process Communication (IPC)

Different from Intents:

- Faster, lower level
- Allows for bidirectional client-server communication

There are many "Managers" on Android phones: Activity, Package, Telephony, Resource, Location, Notification, etc...

Android Permission System

Android framework defines a long list of permissions

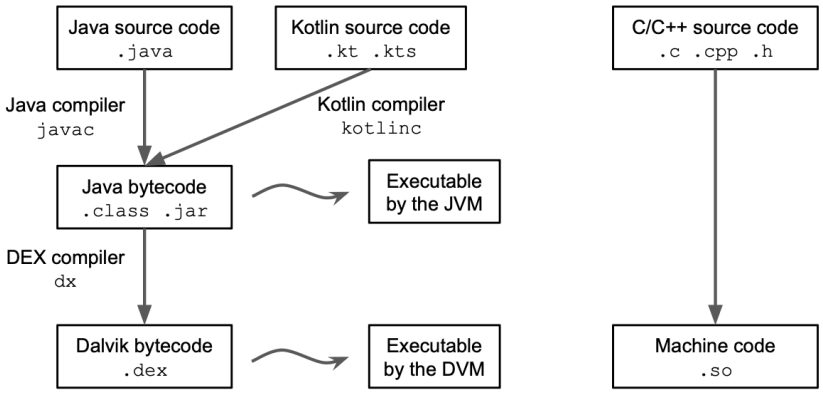
Each of these "protects" security-sensitive capabilities:

- The ability to "do" something sensitive, e.g., open Internet connection, send SMS
- The ability to "access" sensitive information, e.g., location, user contacts, ...

Examples of Permissions: `INTERNET`, `SET_WALLPAPER`, `BLUETOOTH`, ...

Reversing Android Apps

The compilation process

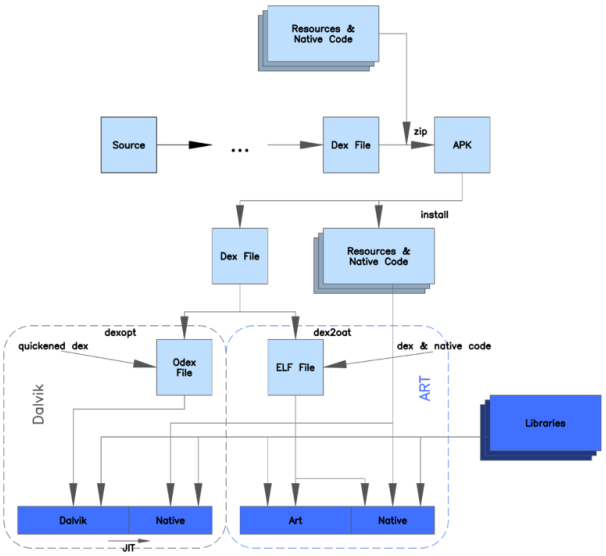


Android Application Package (APK)

An APK is a zip file (with a magic format). Its content is:

- `AndroidManifest.xml` (compressed)
- `classes.dex` (raw Dalvik bytecode)
- `resources.arsc` (compressed)
- `rest/*.xml` (compressed)

The big picture



How to Decompile

All-in-one tools

- Jadx (top tool, check out first)
- JEB (commercial, VERY expensive)
- BytecodeViewer (pretty good one)
- New: Ghidra, open source tool developed by NSA

Using a Java decompiler (java bytecode ⇒ Java)

- Dalvik bytecode ⇒ Java bytecode, e.g., Dex2jar
- Java bytecode ⇒ Java source code, e.g., Jd-GUI

How people find bugs: *decompiling* apps and *analyzing* them

Android Security Mitigations

Android Implements Defense in Depth

Multiple layers of security controls are placed throughout the system. It prevents single vulnerabilities from leading to compromise of the OS or other apps

- **Application Sandbox**: Apps are isolated from each other by being assigned to different users. The sandbox is in the kernel: native code can't bypass it. To bypass the sandbox, an attacker would need to compromise Linux. **Chain of 2 bugs!**
- **SELinux (Mandatory Access Control)**: Security-Enhanced Linux: a Linux kernel security model
  - Useful to defined MAC security polices
  - Example: allow/prevent execution of syscall X
  - "Deny by default" policy

  All processes labeled with the same domain are treated equal. Currently there are 60+ domains in Android
- **ARM MTE (Memory Tagging Extension**: Add a "tag" in the unused, upper bits of a pointer. The tag defines which memory area the pointer is allowed to target. Every load/store is checked, raise exception if pointer goes out of bounds. Mitigates against: buffer overflows, use-after-free. Available since Android 12 on ARMv9 compatible CPUs.
- **Scudo (Hardened Heap Memory Allocator**: Improved version of malloc with some additional features.
  - Randomized allocations
  - Guard pages to prevent some heap overflows
  - More information stored in chunk headers to detect some types of double frees

However, this mitigation has been bypassed multiple times, it only slightly complicates exploitation. Enabled since Android 11.

- etc... Nowadays, to obtain **complete control** over a mobile phone, you need to chain **5-6 bugs**.

Android vs. iOS

Android

- $\sim$ Open source
- Customizable
- Targeting everyone

Strong candidate but...

iOS

- Corporate
- Closed, conservative
- Targeting the rich

Somehow still wins...

In **Switzerland**, iOS is leading the market as of **India**, Android is dominant (95%)

Apple's Strategy

Apple's iOS ecosystem is ***extremely closed***

- iOS is closed source
- iOS and iOS apps can only run on Apple devices
- *You can install apps only from the Apple Store* (changing due to EU legislations)
- Apps need to be signed from the developer *and* Apple
- Not easy to "jailbreak" iOS devices anymore

How come they are still around?

- They were the first $\Rightarrow$ significant chunk of market share
- They make great products, and people know about it

Google's Strategy

Android Inc. (purchased by Google in '05) started developing Android '03

Google needed to catch up: "Open Handset Alliance" (84 companies)

Ecosystem is much ***more*** open in principle

- Android / AOSP is open source
- Android can run on many different devices, even non-Google ones
- Easy to inspect Android apps / reverse them / modify them
- Easy to install apps you develop ("side loading")

- Developers can do many more things
- Quite easy to ”jailbreak” Android devices

Android is becoming closed like iOS

Ecosystem *was* more open

- AOSP is open source but many critical components are not (e.g., DRM libraries or vendor specific components)
- Many obfuscated apps (e.g., Whatsapp)
- Jailbreaking devices has become hard (as vendors force you to jump through weird obstacles, e.g., Xiaomi requires creating an account)
- Many apps refuse to work with a ”customized” device (e.g., Google services, bank apps, etc)

Open vs closed source

Kerckhoff's principle: a cryptosystem should be secure, even if everything about the system, except the key, is public knowledge! (Cryptographic point of view)

	Android	iOS
Open-source (base) OS?	✓	✗
Can OS run on non-Google/Apple device?	✓	✗
Can you run custom OS?	✓	✗
Can you sideload apps?	✓	✗
Can you run custom/modified apps?	✓	✗
Is it easy to tinker with apps?	✓	✗
Easy access to emulator?	✓	✗

The mobile exploit market

Mobile exploit payouts are exploding!

Full remote chain on Samsung S24: only 50,000$ (2024). Which is really low as bounty.

Cellebrite iPhone support (to dump file system)

They have compatible versions with latests versions and iPhones which require AFU = After First Unlock (i.e., is the encryption key loaded in RAM?) iPhone support (to dump file system)

They have compatible versions with latests versions and iPhones which require AFU = After First Unlock (i.e., is the encryption key loaded in RAM?)

If you are in trouble, turn off your phone!

GrapheneOS

Custom Android distro, ultra-hardened and privacy focused

- Extremely reduced attack surface (e.g., way less USB features)
- Actually hardened malloc
- Prevent VPN leaks
- Hardened PDF viewer
- No Google apps/play services
- No zygote (users ASLR/canaries)

iOS Lockdown mode

When Lockdown Mode is enabled, some apps and features will function differently. It protects your device in many ways...

What is the US president using?

- Hyypervisor-based Android distro
- Almost no details are public
- Price not reported (i.e., very expensive)